

LAYERZERO - SECURITY REVIEW

LayerZero Console.

A consolidated security review of LayerZero Console.

CLIENT	AUDITORS	MAIN REVIEW	UPDATES REVIEW	FINAL REPORT
LayerZero	Owen Thurm, Daniel Gelfand, Minato Namikazi, Wafflemakr, Jakub Zmyslowski	January 19 - March 25, 2026	July 16 - July 21, 2026	July 31, 2026

§ 00

Contents.

97 pages - overview, confidence ranking, scope, 80 findings, disclaimer, and about Guardian.

§ 01	Overview	p. 03
------	----------	-------

§ 02	Confidence Ranking	p. 04
------	--------------------	-------

§ 03	Scope & Classifications	p. 05
------	-------------------------	-------

Main Review Scope	p. 06
-------------------	-------

Updates Review Scope	p. 07
----------------------	-------

§ 04	Findings	p. 08
------	----------	-------

Round 1 - Main Review · 68 findings	p. 16-83
-------------------------------------	----------

Round 2 - Remediation Review · 6 findings	p. 84-89
---	----------

Round 3 - Remediation Review · 2 findings	p. 90-91
---	----------

Round 4 - Updates Review · 4 findings	p. 92-95
---------------------------------------	----------

§ 05	Disclaimer	p. 96
------	------------	-------

§ 06	About Guardian	p. 97
------	----------------	-------

Overview.

Guardian reviewed LayerZero Console across three original review rounds from January 19 through March 25, 2026, followed by a Console EVM Updates review from July 16 through July 21 and validation on July 24. The reviewed packages were subsequently folded into LayerZero-Labs/monorepo-external. The consolidated report includes 2 Medium findings, 3 Low findings, 75 Info findings.

CLIENT	LayerZero	REPOSITORIES	Original: monorepo-external · Updates: audit-external
PREPARED BY	Owen Thurm, Daniel Gelfand, Minato Namikazi, Wafflemakr, Jakub Zmyslowski	ORIGINAL FINAL COMMIT	9a1436b0dad246e67dc0f44070091288dabd255c
		UPDATES REVIEW COMMIT	6113ab299c04457e8fcc539e6214ba6b218c34c2
MAIN REVIEW	January 19 - March 25, 2026	LATEST UPDATES COMMIT	0ac5096c55c93a010b681955457868fa860b42c4
UPDATES REVIEW	July 16 - July 21, 2026	EXTERNALIZATION COMMIT	135db014bd9c7e778a2282618cdab24ed6b8fff6

1.1 Confidence Ranking

5/5

Guardian assigns a Confidence Ranking of 5 out of 5. No High or Critical findings were identified across the original engagement or updates review. The two Medium findings and all three Low findings were resolved, with remaining residual risk represented by acknowledged and partially resolved Informational findings.

Very High Confidence

1 V. LOW	2 LOW	3 MED	4 HIGH	5 V. HIGH
-------------	----------	----------	-----------	--------------

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	IN PROG	DECLINED	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0	0	0
High	0	0	0	0	0	0	0
Medium	2	0	0	0	0	0	2
Low	3	0	0	0	0	0	3
Info	75	0	0	0	65	1	9

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong. Recommendation: Suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.
3 Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas such as weak modularity or risky patterns. No critical design flaws were identified, though some patterns could lead to edge-case issues. Recommendation: Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1-2 High/Critical findings per engagement week.
2 Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. Recommendation: Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week, or additional unresolved High/Critical findings uncovered in remediation review.
1 Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥ 5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. Recommendation: Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥ 5 High/Critical findings and overall systemic flaws.

Scope & Classifications.

Guardian reviewed LayerZero Console across the original three-round engagement and the July 2026 Console EVM Updates review. Following the review, the packages were folded into `LayerZero-Labs/monorepo-external` at commit `135db014bd9c7e778a2282618cdab24ed6b8fff6` . The file list below maps the reviewed scope to its externalized paths; the validated review boundary remains the original `audit-external` commits.

3.1 Severity Matrix

	IMP · HIGH	IMP · MEDIUM	IMP · LOW	IMPACT · HIGH	Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.
LIK · HIGH	Critical	High	Medium	IMPACT · MEDIUM	Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.
LIK · MED	High	Medium	Low	IMPACT · LOW	Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.
LIK · LOW	Medium	Low	Low	LIKELIHOOD · HIGH	Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.
				LIKELIHOOD · MEDIUM	Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.
				LIKELIHOOD · LOW	Unlikely to ever occur in production.

§ 03

Main Review Scope.

Contracts and interface sets listed in the original January-March report. Line counts are intentionally omitted.

LayerZero-Labs/monorepo-external (1/2)

36 FILES

LayerZero-Labs/monorepo-external (2/2)

```
Console OFT
ERC20Plus.sol
OFTBurnMint.sol
OFTBurnSelfMint.sol
OFTLockUnlock.sol
OFTNative.sol
OFTBurnMintAlt.sol
OFTBurnSelfMintAlt.sol
OFTLockUnlockAlt.sol

Nexus
Nexus.sol
NexusAlt.sol
NexusERC20.sol
NexusERC20Guard.sol
NexusOFT.sol
NexusOFTAlt.sol
NexusMsgCodec.sol

Nexus modules
NexusFeeConfigModule.sol
NexusModule.sol
NexusPauseModule.sol
NexusRateLimiterModule.sol

Nexus extensions
OFTRegistryBaseUpgradeable.sol
OFTRegistryRBACUpgradeable.sol
TokenScalesBaseUpgradeable.sol
```

```
Common utilities
RateLimiterBaseUpgradeable.sol
RateLimiterRBACUpgradeable.sol
PauseByIDBaseUpgradeable.sol
PauseByIDRBACUpgradeable.sol
EnumerableSetPagination.sol
FeeConfigBaseUpgradeable.sol
FeeConfigRBACUpgradeable.sol
FeeAccountingBaseUpgradeable.sol
FeeAccountingRBACUpgradeable.sol
AllowListBaseUpgradeable.sol
AllowListRBACUpgradeable.sol
AccessControl2StepUpgradeable.sol
PauseBaseUpgradeable.sol
PauseRBACUpgradeable.sol

Interfaces
Multiple interface files
```

§ 03

Updates Review Scope.

Contracts and source files included in the July Console EVM Updates review, remapped to their externalized monorepo paths.

LayerZero-Labs/monorepo-external (1/2)

67 FILES TOTAL

```
apps/oapp-app/contracts/evm/non-upgradeable/contracts/oapp/libs
  OptionsBuilder.sol
apps/oapp-app/contracts/evm/upgradeable/contracts/oapp/alt
  OAppAltUpgradeable.sol
apps/oapp-app/contracts/evm/upgradeable/contracts/oapp/messaging-channel
  OAppMessagingChannelBaseUpgradeable.sol
  OAppMessagingChannelRBACUpgradeable.sol
apps/oapp-app/contracts/evm/upgradeable/contracts/oapp/msg-inspection
  OAppMsgInspectionBaseUpgradeable.sol
  OAppMsgInspectionRBACUpgradeable.sol
apps/oapp-app/contracts/evm/upgradeable/contracts/oapp
  OAppCoreBaseUpgradeable.sol
  OAppCoreRBACUpgradeable.sol
  OAppReceiverUpgradeable.sol
  OAppSenderUpgradeable.sol
  OAppUpgradeable.sol
apps/oapp-app/contracts/evm/upgradeable/contracts/oapp/options-type-3
  OAppOptionsType3BaseUpgradeable.sol
  OAppOptionsType3RBACUpgradeable.sol
apps/oft-app/contracts/evm/non-upgradeable/contracts/libs
  OFTComposeMsgCodec.sol
  OFTMsgCodec.sol
apps/oft-app/contracts/evm/non-upgradeable/contracts/utils
  OFTDecimalUtils.sol
apps/oft-app/contracts/evm/upgradeable/contracts/extended/alt
  OFTBurnMintExtendedRBACAltUpgradeable.sol
  OFTBurnSelfMintExtendedRBACAltUpgradeable.sol
  OFTLockUnlockExtendedRBACAltUpgradeable.sol
apps/oft-app/contracts/evm/upgradeable/contracts/extended
  OFTBurnMintExtendedRBACUpgradeable.sol
  OFTBurnSelfMintExtendedRBACUpgradeable.sol
  OFTCoreExtendedRBACUpgradeable.sol
  OFTLockUnlockExtendedRBACUpgradeable.sol
  OFTNativeExtendedRBACUpgradeable.sol
apps/oft-app/contracts/evm/upgradeable/contracts/oft
  OFTCoreBaseUpgradeable.sol
  OFTCoreRBACUpgradeable.sol
apps/project-types/console-oft-app/contracts/evm/contracts/alt
  OFTBurnMintAlt.sol
  OFTBurnSelfMintAlt.sol
  OFTLockUnlockAlt.sol
apps/project-types/console-oft-app/contracts/evm/contracts
  ERC20Plus.sol
  OFTBurnMint.sol
  OFTBurnSelfMint.sol
  OFTLockUnlock.sol
  OFTNative.sol
```

LayerZero-Labs/monorepo-external (2/2)

```
apps/project-types/nexus-app/contracts/evm/contracts/extensions
  OFTRegistryBaseUpgradeable.sol
  OFTRegistryRBACUpgradeable.sol
  TokenScalesBaseUpgradeable.sol
apps/project-types/nexus-app/contracts/evm/contracts/libs
  NexusMsgCodec.sol
apps/project-types/nexus-app/contracts/evm/contracts/modules
  NexusFeeConfigModule.sol
  NexusModule.sol
  NexusPauseModule.sol
  NexusRateLimiterModule.sol
apps/project-types/nexus-app/contracts/evm/contracts
  Nexus.sol
  NexusAlt.sol
  NexusERC20.sol
  NexusERC20Guard.sol
  NexusOFT.sol
  NexusOFTAlt.sol
apps/proxy-app/contracts/evm/proxy-v5-6-evm/contracts
  ProxyAdmin2Step.sol
  TransparentUpgradeableProxy2Step.sol
contracts/common/utils/evm/upgradeable/contracts/access
  AccessControl2StepUpgradeable.sol
contracts/common/utils/evm/upgradeable/contracts/allowlist
  AllowListBaseUpgradeable.sol
  AllowListRBACUpgradeable.sol
contracts/common/utils/evm/upgradeable/contracts/credit-redirect
  CreditRedirectBaseUpgradeable.sol
  CreditRedirectRBACUpgradeable.sol
contracts/common/utils/evm/upgradeable/contracts/fee-accounting
  FeeHandlerBaseUpgradeable.sol
  FeeHandlerRBACUpgradeable.sol
contracts/common/utils/evm/upgradeable/contracts/fee-config
  FeeConfigBaseUpgradeable.sol
  FeeConfigRBACUpgradeable.sol
contracts/common/utils/evm/upgradeable/contracts/libs
  EnumerableSetPagination.sol
contracts/common/utils/evm/upgradeable/contracts/pause-by-id
  PauseByIdBaseUpgradeable.sol
  PauseByIdRBACUpgradeable.sol
contracts/common/utils/evm/upgradeable/contracts/pause
  PauseBaseUpgradeable.sol
  PauseRBACUpgradeable.sol
contracts/common/utils/evm/upgradeable/contracts/rate-limiter/libs
  RateLimiterUtils.sol
contracts/common/utils/evm/upgradeable/contracts/rate-limiter
  RateLimiterBaseUpgradeable.sol
  RateLimiterRBACUpgradeable.sol
```

§ 04

Findings.

80 findings across the original review and updates phase: 2 Medium findings, 3 Low findings, 75 Info findings. Findings are grouped by review round, with visible IDs resetting inside each round.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 1 - MAIN REVIEW					
M-01	Nexus composeFrom Is Not Original Sender	Logical Error	MEDIUM	RESOLVED	p. 16
M-02	Missing Overflow Validation	Validation	MEDIUM	RESOLVED	p. 17
L-01	quoteOFT Doesn't Account For Fees	Warning	LOW	RESOLVED	p. 18
L-02	quoteOFT Misses outboundEnabled	Logical Error	LOW	RESOLVED	p. 19
I-01	Some Tokens Cannot Bridge Back Full Supply	Warning	INFO	ACKNOWLEDGED	p. 20
I-02	IMintableBurnable Return Values Ignored	Warning	INFO	ACKNOWLEDGED	p. 21
I-03	Pausing Asymmetry Between Debit And Credit	Documentation	INFO	ACKNOWLEDGED	p. 22
I-04	Misleading Fee Balance View Function	Warning	INFO	ACKNOWLEDGED	p. 23
I-05	Allowlist Not Enforced On Approvals And Permit	Warning	INFO	ACKNOWLEDGED	p. 24
I-06	Approvals And Permit Usable During Global Pause	Warning	INFO	ACKNOWLEDGED	p. 25
I-07	Owner Must Pass Downscaled Value	Documentation	INFO	ACKNOWLEDGED	p. 26

§ 04

Findings, continued.

Index page 2 of 8.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 1 - MAIN REVIEW					
I-08	Closed Rate Limit Allows Zero-Amount Messaging	Warning	INFO	ACKNOWLEDGED	p. 27
I-09	Withdraw Fees May Drain Balance On Override	Logical Error	INFO	ACKNOWLEDGED	p. 28
I-10	Incorrect Registry Storage Location	Documentation	INFO	RESOLVED	p. 29
I-11	Allowlist Mode Switching Preserves Inactive Set	Warning	INFO	ACKNOWLEDGED	p. 30
I-12	Fee Withdrawal Blocked By Allowlist Settings	Documentation	INFO	ACKNOWLEDGED	p. 31
I-13	Allowlist Can Block Fund Recovery	Warning	INFO	ACKNOWLEDGED	p. 32
I-14	Upgradeable Immutables Can Change Behavior	Warning	INFO	ACKNOWLEDGED	p. 33
I-15	Exempt Users Can Reduce Net Accounting	Warning	INFO	ACKNOWLEDGED	p. 34
I-16	Zero Amounts Cause Rate Limit Precision Loss	Rounding	INFO	ACKNOWLEDGED	p. 35
I-17	Rate Limits Consumed DoS	Warning	INFO	RESOLVED	p. 36
I-18	Lacking Configuration List Duplicate Check	Validation	INFO	ACKNOWLEDGED	p. 37

§ 04

Findings, continued.

Index page 3 of 8.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 1 - MAIN REVIEW					
I-19	Nexus And Console Tokens Should Have Low Supply	Warning	INFO	ACKNOWLEDGED	p. 38
I-20	Unlimited Transfers With Net Accounting	Logical Error	INFO	ACKNOWLEDGED	p. 39
I-21	Non-exempt Users May Bypass Inbound Rate Limits	Warning	INFO	ACKNOWLEDGED	p. 40
I-22	De-registration Risk	Warning	INFO	ACKNOWLEDGED	p. 41
I-23	Minting Bypasses Global Pause	Warning	INFO	ACKNOWLEDGED	p. 42
I-24	Recover Bypasses Validations	Warning	INFO	ACKNOWLEDGED	p. 43
I-25	Missing InvalidLocalDecimals Error	Validation	INFO	RESOLVED	p. 44
I-26	Incorrect Max Amount Reported	Unexpected Behavior	INFO	ACKNOWLEDGED	p. 45
I-27	Enforced Options Cannot Be Applied Individually	Compatibility	INFO	ACKNOWLEDGED	p. 46
I-28	Some Tokens Incompatible	Compatibility	INFO	ACKNOWLEDGED	p. 47
I-29	Lacking ERC7802 Compliance	Compatibility	INFO	ACKNOWLEDGED	p. 48

§ 04

Findings, continued.

Index page 4 of 8.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 1 - MAIN REVIEW					
I-30	Trapped ETH Through IzReceive	Warning	INFO	ACKNOWLEDGED	p. 49
I-31	Redundant Sources Of Truth	Best Practices	INFO	RESOLVED	p. 50
I-32	quoteSend Ignores Pausability Or Rate Limits	Warning	INFO	ACKNOWLEDGED	p. 51
I-33	Breaking MsgInspector Interface Update	Compatibility	INFO	ACKNOWLEDGED	p. 52
I-34	Mints Can Occur While Token Is Paused	Documentation	INFO	ACKNOWLEDGED	p. 53
I-35	Payouts Spend Fees From Same Pool	Warning	INFO	ACKNOWLEDGED	p. 54
I-36	Scale Decimals Are Not Exposed	Compatibility	INFO	PARTIALLY RESOLVED	p. 55
I-37	quoteOFT Does Not Consult Guard	Unexpected Behavior	INFO	ACKNOWLEDGED	p. 56
I-38	Initialize Must Be Invoked During Deployment	Best Practices	INFO	ACKNOWLEDGED	p. 57
I-39	Nexus Doesn't Support Native Inner Tokens	Suggestion	INFO	ACKNOWLEDGED	p. 58
I-40	New Network Supports Allows Circumvention	Warning	INFO	ACKNOWLEDGED	p. 59

§ 04

Findings, continued.

Index page 5 of 8.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 1 - MAIN REVIEW					
I-41	Lacking Pause Idempotency Check	Best Practices	INFO	ACKNOWLEDGED	p. 60
I-42	Event Log Manipulation With Reentrancy	Events	INFO	ACKNOWLEDGED	p. 61
I-43	NexusAlt Lacks Sanity Check	Validation	INFO	RESOLVED	p. 62
I-44	Missing MinterBurnerCallFailed Implementation	Superfluous Code	INFO	RESOLVED	p. 63
I-45	Mint/Burn Result Ignored By OFTBurnMint	Compatibility	INFO	ACKNOWLEDGED	p. 64
I-46	Credit Can Consume An Unexpected Amount Of Gas	DoS	INFO	ACKNOWLEDGED	p. 65
I-47	Unsynchronized Limits Can Lead To Stuck Funds	Warning	INFO	ACKNOWLEDGED	p. 66
I-48	Rate Limit Bypass Through Exempt Address	Warning	INFO	ACKNOWLEDGED	p. 67
I-49	Fee-on-Transfer/Rebasing Tokens Not Supported	Warning	INFO	ACKNOWLEDGED	p. 68
I-50	Tokens With Inconsistent Decimals Incompatible	Compatibility	INFO	ACKNOWLEDGED	p. 69
I-51	OFTNative Stuck Message Risk	Warning	INFO	ACKNOWLEDGED	p. 70

§ 04

Findings, continued.

Index page 6 of 8.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 1 - MAIN REVIEW					
I-52	Missing Events	Events	INFO	RESOLVED	p. 71
I-53	Rate Limit Quotes Can Be Misleading	Unexpected Behavior	INFO	ACKNOWLEDGED	p. 72
I-54	Outside Tokens Do Not Follow The Guard	Compatibility	INFO	ACKNOWLEDGED	p. 73
I-55	Fee Rounds Down	Rounding	INFO	ACKNOWLEDGED	p. 74
I-56	Quote Misses Underlying Token Pause State	Unexpected Behavior	INFO	ACKNOWLEDGED	p. 75
I-57	Fees Inflated For Small Sends	Rounding	INFO	ACKNOWLEDGED	p. 76
I-58	Rate Limits Retroactively Affected	Unexpected Behavior	INFO	ACKNOWLEDGED	p. 77
I-59	Net Accounting Abused For One Sided Rate	Rounding	INFO	ACKNOWLEDGED	p. 78
I-60	Rate Limit Configurations Sandwiched	Frontrunning	INFO	ACKNOWLEDGED	p. 79
I-61	Incorrect To Address Used	Logical Error	INFO	ACKNOWLEDGED	p. 80
I-62	Default Rate Limit Cannot Apply Across Tokens	Compatibility	INFO	ACKNOWLEDGED	p. 81

§ 04

Findings, continued.

Index page 7 of 8.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 1 - MAIN REVIEW					
I-63	nexusQuoteOFT Misses Exemptions	Warning	INFO	ACKNOWLEDGED	p. 82
I-64	nexusQuoteOFT Does Not Consult The MsgInspector	Unexpected Behavior	INFO	ACKNOWLEDGED	p. 83
ROUND 2 - REMEDIATION REVIEW					
I-01	Stale Checkpointing Documentation	Documentation	INFO	ACKNOWLEDGED	p. 84
I-02	Potential Overflow In maxAmountLD	Warning	INFO	ACKNOWLEDGED	p. 85
I-03	AccessControl2StepUpgradeable Lacks Timelock	Warning	INFO	ACKNOWLEDGED	p. 86
I-04	PauseByIDBase._setPaused Lacks Idempotent Check	Warning	INFO	ACKNOWLEDGED	p. 87
I-05	Missing AccessControl Init Locks Admin Functions	Logical Error	INFO	ACKNOWLEDGED	p. 88
I-06	PreCrime Simulation Layer Entirely Removed	Warning	INFO	ACKNOWLEDGED	p. 89
ROUND 3 - REMEDIATION REVIEW					
I-01	Shared Nonce Channel Enables Token-level DoS	DoS	INFO	ACKNOWLEDGED	p. 90
I-02	Immutable Decimals Can Change On Upgrade	Upgradeability	INFO	ACKNOWLEDGED	p. 91
ROUND 4 - UPDATES REVIEW					
L-01	OFT interface ID omits receiver functions	Unexpected Behavior	LOW	RESOLVED	p. 92

§ 04

Findings, continued.

Index page 8 of 8.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 4 - UPDATES REVIEW					
I-01	Divergent guards can block Nexus credits	Unexpected Behavior	INFO	ACKNOWLEDGED	p. 93
I-02	Empty init data leaves proxy unprotected	Informational	INFO	RESOLVED	p. 94
I-03	Burn docs omit nonce eligibility	Documentation	INFO	RESOLVED	p. 95

MEDIUM **M-01**

RESOLVED

Nexus composeFrom Is Not Original Sender.

CATEGORY Logical Error

LOCATION Nexus.sol

REVIEW Round 1 - Main Review

DESCRIPTION

In Nexus, both compose identity fields resolve to the NexusOFT wrapper instead of the original user. Standard OFT behavior preserves the OFT contract as `_from` and the end user as `composeFrom`, but the Nexus call flow causes `OFTMsgCodec.encode()` to capture the wrapper as the compose initiator. As a result, receiving contracts cannot reliably identify or authenticate the original sender. Any downstream logic that depends on user-specific authorization or accounting sees every compose as coming from the same wrapper address.

RECOMMENDATION

Use Nexus-specific message encoding that writes the original user into `composeFrom` instead of relying on the current `msg.sender` context.

RESOLUTION

Status: Resolved.

MEDIUM **M-02**

RESOLVED

Missing Overflow Validation.

CATEGORY Validation

LOCATION DecimalUtils.sol

REVIEW Round 1 - Main Review

DESCRIPTION

In the DecimalUtils file the `_toSD` does not perform any validation to ensure that the result of the decimal conversion does not overflow a `uint64`. `` `function _toSD(uint256 _amountLD) internal view virtual returns (uint64 amountSD) { return uint64(_amountLD / DECIMAL_CONVERSION_RATE); }` `` As a result, large crosschain transfers for tokens with a high supply can result in the sender burning a huge amount of tokens and receiving a very small amount of tokens out, representing a complete loss of assets.

RECOMMENDATION

Add validation on the result of `_amountLD / DECIMAL_CONVERSION_RATE` to ensure that it does not overflow a `uint64` before casting it as such.

RESOLUTION

Status: Resolved.

LOW **L-01**

RESOLVED

quoteOFT Doesn't Account For Fees.

CATEGORY Warning LOCATION Global REVIEW Round 1 - Main Review

DESCRIPTION

The quoteOFT function returns maxAmountLD directly from the rate limiter's available outbound capacity. However, the rate limiter enforces limits on amountReceivedLD (post-fee amount), while maxAmountLD is meant to represent the maximum amountSentLD (pre-fee amount) a user can submit. When fees are configured, users can actually send more than the reported maximum. For

example, with a rate limit of 1000 and a 10% fee, users can send approximately 1111 tokens (resulting in 1000 received), but quoteOFT reports a maximum of 1000. This causes UIs and integrations relying on this value to understate available capacity.

RECOMMENDATION

Consider accounting for the fee in quoteOFT for the maxAmountLD calculation.

RESOLUTION

Status: Resolved.

LOW L-02

RESOLVED

quoteOFT Misses outboundEnabled.

CATEGORY Logical Error

LOCATION Global

REVIEW Round 1 - Main Review

DESCRIPTION

The nexusQuoteOFT and quoteOft on the Console uses the maxAmountLD as presented by the getRateLimitUsages function. However this function does not account for when the config bitmap of the rate limit indicates that the outbound rate limit is disabled. As a result the nexusQuoteOFT function will present inaccuracies in the maxAmountLD when the outboundEnabled config value for a rate limit is false.

RECOMMENDATION

Take into consideration whether the outbound pathway is enabled or not for the rate limit in the quote functions.

RESOLUTION

Status: Resolved.

INFO I-01

ACKNOWLEDGED

Some Tokens Cannot Bridge Back Full Supply.

CATEGORY Warning LOCATION Global REVIEW Round 1 - Main Review

DESCRIPTION

Due to the way the fee logic works, in some cases it is impossible for a networks supply to go entirely to zero while a fee is configured. Consider the following example:

- A token uses a lockbox OFT on Network A, and a a MintBurn OFT on Network B
- Network B has an outgoing fee of 50 BIPs and a token supply of X, backed by X tokens on Network A in the lockbox
- Token on Network B has 18 decimals and 6 shared decimals
- Even if all X tokens are transferred back to Network A, a fee of $50X/10000$ will be taken and kept on Network B
- Since the dust removal logic favors counting dust removed as fees, a remaining amount that is below the conversion rate will just simply not be transferrable back
- Such a dust amount is guaranteed to always exist as long as the fee does not get truncated to zero

RECOMMENDATION

Be aware that a dust supply of a token will be trapped on a network unless the fee is removed.

RESOLUTION

Status: Acknowledged.

INFO I-02

ACKNOWLEDGED

IMintableBurnable Return Values Ignored.

CATEGORY Warning

LOCATION Nexus.sol: 272, 388

REVIEW Round 1 - Main Review

DESCRIPTION

The IMintableBurnable interface shows that both burn() and mint() return bool success. However, in Nexus.sol, these return values are ignored. If the underlying token implementation returns false instead of reverting, the operations will fail silently, leading to:

- Tokens not actually burned but message sent (unbacked mint on destination)
- Tokens not actually minted but message processed (user doesn't receive funds)

RECOMMENDATION

Check return values or document that tokens that revert on failure should be used.

RESOLUTION

Status: Acknowledged.

INFO I-03

ACKNOWLEDGED

Pausing Asymmetry Between Debit And Credit.

CATEGORY Documentation

LOCATION Nexus.sol: 376

REVIEW Round 1 - Main Review

DESCRIPTION

The whenNotPaused(_nexusId) modifier appears only on _debit, which is the send path. The _credit function used for receiving tokens has no pause check. This means that pausing stops outbound transfers but allows inbound which may be unexpected.

RECOMMENDATION

Document this asymmetry clearly.

RESOLUTION

Status: Acknowledged.

INFO I-04

ACKNOWLEDGED

Misleading Fee Balance View Function.

CATEGORY Warning

LOCATION Nexus.sol: 404

REVIEW Round 1 - Main Review

DESCRIPTION

The Nexus's fees accumulated is simply calculated as `IERC20(_token).balanceOf(address(this))`; within function `getFeeBalance`. It should be clearly noted to users that if tokens are directly sent to the Nexus, they'll be counted as withdrawable fees.

RECOMMENDATION

Clearly document this behavior.

RESOLUTION

Status: Acknowledged.

INFO I-05

ACKNOWLEDGED

Allowlist Not Enforced On Approvals And Permit.

CATEGORY Warning

LOCATION ERC20Plus.sol

REVIEW Round 1 - Main Review

DESCRIPTION

The allowlist checks are only applied to token movement functions (transfer/transferFrom/burn) but not to approval state changes. Blacklisted or non-whitelisted users can set or increase allowances through functions approve/increaseAllowance/decreaseAllowance and permit while disallowed by the allowlist. These allowances can be executed immediately once the owner/spender becomes allowlisted.

RECOMMENDATION

Clearly document that approval changes do not verify allowlist.

RESOLUTION

Status: Acknowledged.

INFO I-06

ACKNOWLEDGED

Approvals And Permit Usable During Global Pause.

CATEGORY Warning

LOCATION ERC20Plus.sol

REVIEW Round 1 - Main Review

DESCRIPTION

Functions such as approve and permit are not gated by whenNotGloballyPaused. While transfers are paused, actors can set allowances and signatures that can be executed immediately after unpause, which should be documented.

RECOMMENDATION

Clearly document this behavior.

RESOLUTION

Status: Acknowledged.

INFO I-07

ACKNOWLEDGED

Owner Must Pass Downscaled Value.

CATEGORY Documentation LOCATION RateLimiterUpgradeable.sol REVIEW Round 1 - Main Review

DESCRIPTION

Rate-limit configs/states expect downscaled units when SCALE_DECIMALS > 0, but only getRateLimitUsages upscales for display, so an admin passing base decimal limits will silently set much larger caps and lead to a misconfigured limit.

RECOMMENDATION

Add explicit documentation on IRateLimiterCore setters and RateLimiterUpgradeable admin functions stating inputs are downscaled units.

RESOLUTION

Status: Acknowledged.

INFO I-08

ACKNOWLEDGED

Closed Rate Limit Allows Zero-Amount Messaging.

CATEGORY Warning

LOCATION RateLimiterCoreUpgradeable.sol

REVIEW Round 1 - Main Review

DESCRIPTION

Setting the rate limit to closed (limit = 0) blocks non-zero transfers, but zero-amount sends still pass the rate-limit checks. This allows messages (including optional compose payloads) to be emitted even when operators expect the rate limiter to fully halt activity. If downstream logic assumes that "closed" means no cross-chain messages, this creates an unexpected bypass for message-only actions.

RECOMMENDATION

If the intended behavior is to block all messaging when closed, add an explicit amount > 0 check in the send path. Otherwise, document that "closed" only blocks value transfer and that pause must be used to halt messaging.

RESOLUTION

Status: Acknowledged.

INFO I-09

ACKNOWLEDGED

Withdraw Fees May Drain Balance On Override.

CATEGORY Logical Error

LOCATION FeeCoreUpgradeable.sol: 166

REVIEW Round 1 - Main Review

DESCRIPTION

FeeCoreUpgradeable comments suggest overrides may track fees via `balanceOf(this)` instead of `storage`. That pattern is safe for mint/burn OFTs, but not for lock/unlock OFTs where the contract holds both user deposits and accrued fees. If an override followed the documentation literally in a lock/unlock setup, `withdrawFees()` could treat the full contract balance as fees and drain user funds along with the real fee balance. Current implementations are safe, but the guidance is too broad.

RECOMMENDATION

Clarify in the `getFeeBalance()` and `_resetFeeBalance()` documentation that balance-based fee tracking is only safe when contract balances do not also include user-held liquidity.

RESOLUTION

Status: Acknowledged.

INFO I-10

RESOLVED

Incorrect Registry Storage Location.

CATEGORY Documentation

LOCATION OFTRegistryCoreUpgradeable.sol

REVIEW Round 1 - Main Review

DESCRIPTION

In the OFTRegistryCoreUpgradeable contract the OFT_REGISTRY_CORE_STORAGE_LOCATION is stored as 0x5e17fe86d24c27c5744e7e62d5c0e4b546c352da7d7ccf57d4d9d4da22a06900 with a comment indicating it is computed as `keccak256(abi.encode(uint256(keccak256("layerzerov2.storage.oftregistrycore")) - 1)) & ~bytes32(uint256(0xff))`. However `keccak256(abi.encode(uint256(keccak256("layerzerov2.storage.oftregistrycore")) - 1)) & ~bytes32(uint256(0xff))` evaluates to 0x3061291cb11960a6131e0dc3984d775ee9ea1d64c1334ba80fc584e9cc0bf400 which does not match the storage location used.

RECOMMENDATION

Correct the comment or the storage location bytes.

RESOLUTION

Status: Resolved.

INFO I-11

ACKNOWLEDGED

Allowlist Mode Switching Preserves Inactive Set.

CATEGORY Warning

LOCATION AllowlistCore.sol

REVIEW Round 1 - Main Review

DESCRIPTION

When switching between Blacklist and Whitelist modes via `setAllowlistMode()`, the contract only updates the active mode but does not clear the inactive set. Both `blacklistedSet` and `whitelistedSet` persist in storage regardless of the current mode. If the owner toggles modes and later switch back, previously configured entries immediately become active again. For example, blacklisting users, switching to Whitelist mode, then returning to Blacklist mode will re-activate all previous blacklist entries without additional transactions.

RECOMMENDATION

Clearly document this behavior.

RESOLUTION

Status: Acknowledged.

INFO I-12

ACKNOWLEDGED

Fee Withdrawal Blocked By Allowlist Settings.

CATEGORY Documentation LOCATION Global REVIEW Round 1 - Main Review

DESCRIPTION

In whitelist or blacklist modes, ERC20Plus.transfer and NexusERC20.transfer requires the caller and recipient to be allowlisted. Nexus uses transfer when withdrawing fee balances to a recipient. If address(Nexus) or the recipient is not allowlisted (or is blacklisted), withdrawFees reverts, leaving fee balances stuck in the Nexus contract.

RECOMMENDATION

Clearly document that the Nexus must be allowlisted as well as the recipient.

RESOLUTION

Status: Acknowledged.

INFO I-13

ACKNOWLEDGED

Allowlist Can Block Fund Recovery.

CATEGORY Warning LOCATION Global REVIEW Round 1 - Main Review

DESCRIPTION

ERC20Plus.recoverFunds and NexusERC20.recoverFunds explicitly revert when the _from address is allowlisted. Fee withdrawal from Nexus uses token transfers, which require address(Nexus) to be allowlisted in whitelist mode. As a result, if the protocol allowlists Nexus to enable fee withdrawals, fund recovery from the Nexus address is always blocked by design.

RECOMMENDATION

Clearly document this behavior

RESOLUTION

Status: Acknowledged.

INFO I-14

ACKNOWLEDGED

Upgradeable Immutables Can Change Behavior.

CATEGORY Warning LOCATION Global REVIEW Round 1 - Main Review

DESCRIPTION

Several upgradeable contracts rely on immutable values (e.g., SCALE_DECIMALS, decimal conversion rates). Upgrading to an implementation compiled with different immutables silently changes behavior without storage migration, potentially altering rate-limit math and token decimal handling.

RECOMMENDATION

Document this and ensure deployments are done without changing immutables.

RESOLUTION

Status: Acknowledged.

INFO I-15

ACKNOWLEDGED

Exempt Users Can Reduce Net Accounting.

CATEGORY Warning

LOCATION RateLimiterCoreUpgradeable.sol

REVIEW Round 1 - Main Review

DESCRIPTION

In the `_applyRateLimit` function users who are marked as exempt can avoid the rate limits, as expected. However, when a rate limit has net accounting enabled, an exempt user can abuse this. Consider the following scenario:

- Chain A is the current chain
- Chain A has the inbound and outbound rate limits enabled
- Chain A has net accounting enabled
- Chain A currently has a 5e18 usage on the outbound rate limit, and the maximum is 10e18
- An exempt user bridges out 5e18 tokens and avoids the rate limit
- The exempt user then transfers these tokens to a different address under their control
- The exempt user then bridges them back to Chain A
- Since Chain A uses net accounting, the Chain's outbound rate limit usage is decreased to 0.

RECOMMENDATION

Be aware of this ability from exempt addresses, and consider this carefully for setups that use both exempt addresses and net accounting. All exempt addresses must be trusted entirely.

RESOLUTION

Status: Acknowledged.

INFO I-16

ACKNOWLEDGED

Zero Amounts Cause Rate Limit Precision Loss.

CATEGORY Rounding LOCATION RateLimiterCoreUpgradeable.sol REVIEW Round 1 - Main Review

DESCRIPTION

Zero value sends are allowed, however they do update the rate limit decay which uses round down division. If a smaller limit is used relative to the length of the window for a rate limit, some precision loss can be forced through zero amount transfers maliciously occurring at a low interval to keep the rate limits down marginally.

RECOMMENDATION

Be aware of this rounding and consider disallowing zero value OFT send.

RESOLUTION

Status: Acknowledged.

INFO I-17

RESOLVED

Rate Limits Consumed DoS.

CATEGORY Warning LOCATION Global REVIEW Round 1 - Main Review

DESCRIPTION

It is possible for anyone to purposely consume the rate limits when net accounting is not enabled by simply transferring out and then back into the src chain.

RECOMMENDATION

Be aware of and document this loophole so that those who would like to avoid it can use net accounting.

RESOLUTION

Status: Resolved.

INFO I-18

ACKNOWLEDGED

Lacking Configuration List Duplicate Check.

CATEGORY Validation

LOCATION RateLimiterCoreUpgradeable.sol

REVIEW Round 1 - Main Review

DESCRIPTION

In the RateLimiterCoreUpgradeable contract `_setRateLimitConfigs` and `_setRateLimitStates` configuration functions there is no validation that the `param.id` has not already been configured in the list before. If a parameter with the same id were to be provided it could lead to an unexpected configuration overwrite.

RECOMMENDATION

Be aware of this lacking validation and consider adding it to validate against any potential mishaps during configuration. Otherwise be sure to review configurations to ensure that only unique param ids are being configured.

RESOLUTION

Status: Acknowledged.

INFO I-19

ACKNOWLEDGED

Nexus And Console Tokens Should Have Low Supply.

CATEGORY Warning LOCATION Global REVIEW Round 1 - Main Review

DESCRIPTION

The Nexus and Console implementations use a value of 0 for the `_scaleDecimals` hardcoded in their constructor. As a result, to avoid any unexpected conditions these systems should avoid using tokens what may have a total supply greater than ~79 Billion tokens using 18 decimals.

RECOMMENDATION

Be aware of this limitation and consider it when using tokens with these systems.

RESOLUTION

Status: Acknowledged.

INFO I-20

ACKNOWLEDGED

Unlimited Transfers With Net Accounting.

CATEGORY Logical Error

LOCATION RateLimiterCoreUpgradeable.sol: 326

REVIEW Round 1 - Main Review

DESCRIPTION

When one transfer direction is disabled but `netAccountingEnabled` remains on, users can send unlimited volume through the disabled direction and still reduce usage in the opposite direction. That lets them repeatedly restore capacity on the enforced side and bypass the intended rate limit. For example, if outbound is disabled, inbound is limited, and net accounting is enabled, a user can send tokens outbound without restriction, reduce inbound usage, then immediately consume the restored inbound capacity. Repeating the cycle turns a configured limit into effectively unlimited throughput.

RECOMMENDATION

Document this behavior clearly so operators understand that disabling one direction can still affect the opposite direction when net accounting is enabled.

RESOLUTION

Status: Acknowledged.

INFO I-21

ACKNOWLEDGED

Non-exempt Users May Bypass Inbound Rate Limits.

CATEGORY Warning

LOCATION RateLimiterCoreUpgradeable.sol

REVIEW Round 1 - Main Review

DESCRIPTION

In the `_applyRateLimit` function the `_user` address provided is used to check whether the user is an exempt address for the rate limit check. On the sending side, this user is the address that initiated the action and sent the tokens. However on the receiving side, this `_user` address represents the receiving side, and anyone can initiate transfers to this address.

RECOMMENDATION

Consider if this is the desired way to check whether a crosschain transfer should be exempt from the rate limit, or if there should be some authentication also based on the authorization of the sender address from the sending chain. Though this introduces some complexity when receiving from a non-EVM compatible network. Generally, be aware of this ability for anyone to transfer to the addresses and contracts that are considered exempt from the inbound rate limits, and document it accordingly.

RESOLUTION

Status: Acknowledged.

INFO I-22

ACKNOWLEDGED

De-registration Risk.

CATEGORY Warning LOCATION Global REVIEW Round 1 - Main Review

— DESCRIPTION

If a token is de-registered while there are in-flight messages, then those messages will get stuck in the channel. If the token is re-registered or another token is registered using the same tokenId, then those stuck messages will be executable again unexpectedly.

— RECOMMENDATION

To avoid any unexpected behavior, consider blacklisting a tokenId after it is de-registered and requiring that a new one is used. Furthermore, handle token registration across all chains with extreme care, as it must be done in a timely manner and with exact accuracy every time.

— RESOLUTION

Status: Acknowledged.

INFO I-23

ACKNOWLEDGED

Minting Bypasses Global Pause.

CATEGORY Warning LOCATION Global REVIEW Round 1 - Main Review

DESCRIPTION

The mint function in both ERC20Plus and NexusERC20 lacks the whenNotGloballyPaused modifier, allowing addresses with the MINTER_ROLE to mint tokens even when the system is globally paused. While transfer, transferFrom, and burn are all gated by pause checks, minting remains unrestricted. Consequently token supply can increase during emergency pause conditions when all other token movement is frozen.

RECOMMENDATION

Clearly document this behavior.

RESOLUTION

Status: Acknowledged.

INFO I-24

ACKNOWLEDGED

Recover Bypasses Validations.

CATEGORY Warning LOCATION Global REVIEW Round 1 - Main Review

DESCRIPTION

The recoverFunds function in ERC20Plus and NexusERC20 validates that the source address is not allowlisted but performs no validation on the destination address. By using super_transfer() instead of the standard transfer function, it bypasses both pause checks and allowlist enforcement on the recipient. This allows a FUND_RECOVERY_ROLE holder to transfer recovered tokens to any address, including those that are blacklisted.

RECOMMENDATION

Clearly document this behavior.

RESOLUTION

Status: Acknowledged.

INFO I-25

RESOLVED

Missing InvalidLocalDecimals Error.

CATEGORY Validation

LOCATION DecimalUtils.sol

REVIEW Round 1 - Main Review

DESCRIPTION

In the DecimalUtils utility contract there is no logic to explicitly validate the local decimals are greater than or equal to the shared decimals. As a result, in the event that the shared decimals is specified to be larger than the local decimals the deployment will revert with a non-descriptive panic revert.

RECOMMENDATION

Consider implementing an explicit validation that the shares decimals is not greater than the local decimals and reverting with a InvalidLocalDecimals error similarly to the canonical OFT implementation.

RESOLUTION

Status: Resolved.

INFO I-26

ACKNOWLEDGED

Incorrect Max Amount Reported.

CATEGORY Unexpected Behavior

LOCATION Nexus.sol

REVIEW Round 1 - Main Review

DESCRIPTION

When the rate limiter is globally disabled, `getRateLimitUsages()` reports `type(uint256).max` as the maximum transferable amount. That is misleading because Nexus amounts are still bounded by OFT shared-decimal constraints, so values anywhere near `uint256.max` are not actually transferable. The canonical OFT implementation reports `type(uint64).max`, and the most accurate bound here is `type(uint64).max * DECIMAL_CONVERSION_RATE`. Returning `uint256.max` therefore overstates what callers can really send.

RECOMMENDATION

Match the canonical OFT behavior by reporting `type(uint64).max`, or return the stricter bound of `type(uint64).max * DECIMAL_CONVERSION_RATE`.

RESOLUTION

Status: Acknowledged.

INFO I-27

ACKNOWLEDGED

Enforced Options Cannot Be Applied Individually.

CATEGORY Compatibility

LOCATION Nexus.sol

REVIEW Round 1 - Main Review

DESCRIPTION

A Nexus instance may support multiple NexusOFTs with different underlying token implementations. One underlying token implementation may have special logic or callback hooks on the destination chain and thus may require more gas than other underlying tokens that are supported by the Nexus. However the combineOptions function does not allow options to be enforced per nexus id, only per Eid. This leaves the Nexus with rigid option enforcement that cannot fully adapt to the various tokens that may be a part of the Nexus.

RECOMMENDATION

Consider implementing a version of combineOptions that allows enforcedOptions to be specified on a nexus id level instead of an Eid level.

RESOLUTION

Status: Acknowledged.

INFO I-28

ACKNOWLEDGED

Some Tokens Incompatible.

CATEGORY Compatibility

LOCATION Global

REVIEW Round 1 - Main Review

DESCRIPTION

The Nexus contract uses an IMintableBurnable interface that expects a boolean return value from the mint and burn functions, however not all tokens who expose these functions return a boolean. For example, the GMX token on Arbitrum implements a MintableBaseToken where the mint and burn functions do not return any return data. These tokens are incompatible out of the box with the system as the invocations of mint and burn will revert due to unexpected lack of returndata. These tokens would need adapters to be correctly supported.

RECOMMENDATION

Consider removing the return values from the IMintableBurnable interface as they are not currently used and instead limit the compatibility of the system.

RESOLUTION

Status: Acknowledged.

INFO I-29

ACKNOWLEDGED

Lacking ERC7802 Compliance.

CATEGORY Compatibility LOCATION Global REVIEW Round 1 - Main Review

DESCRIPTION

The Nexus system implements minting and burning calls on a minterBurner contract meant for crosschain mint/burn actions. There is an EIP7802 which is intended to support exactly this, and is used by the most popular cross-chain stablecoin on OFT rails, USDT0, however the Nexus system does not adhere to this EIP and instead defines it's own IMintableBurnable interface.

RECOMMENDATION

Consider adopting the EIP7802 interface for the Nexus system.

RESOLUTION

Status: Acknowledged.

INFO I-30

ACKNOWLEDGED

Trapped ETH Through IzReceive.

CATEGORY Warning LOCATION Nexus.sol REVIEW Round 1 - Main Review

DESCRIPTION

The IzReceive function does not implement any validation that ensures that the msg.value provided is zero and there are no ETH rescue functions, therefore any message that is executed with nonzero msg.value traps that ETH in the Nexus contract.

RECOMMENDATION

Consider validating that the msg.value is zero in _IzRecieve or adding a rescue native function.

RESOLUTION

Status: Acknowledged.

INFO I-31

RESOLVED

Redundant Sources Of Truth.

CATEGORY Best Practices

LOCATION NexusOFT.sol

REVIEW Round 1 - Main Review

DESCRIPTION

The NexusOFT contract stores a `TOKEN_ID` which is immutable and stored at construction time, however the Nexus stores each OFT by a `tokenId` that is provided by the owner who registers it, with no validation that this id matches the one returned from the `NexusOFT.tokenId` function. This leaves room for incorrect configurations and creates parallel tracking for the same information in the Nexus and NexusOFT contracts.

RECOMMENDATION

Consider either having the NexusOFT contract fetch the `NEXUS.getTokenId(address(this))` result as the implementation for the `tokenId` function (less gas efficient, but deduplicates state), or validating that the reported `tokenId()` on the NexusOFT instance matches the `tokenId` it's being configured for in the `registerToken` function.

RESOLUTION

Status: Resolved.

INFO I-32

ACKNOWLEDGED

quoteSend Ignores Pausability Or Rate Limits.

CATEGORY Warning LOCATION Global REVIEW Round 1 - Main Review

DESCRIPTION

quoteSend returns a valid MessagingFee without checking pause state or rate limits. It calls _debitView which performs pure calculations. However, the actual send calls _debit which has whenNotPaused modifier and enforces rate limits with _outflow. Users can receive valid-looking quotes for sends that will revert.

RECOMMENDATION

Clearly document this behavior to integrators.

RESOLUTION

Status: Acknowledged.

INFO I-33

ACKNOWLEDGED

Breaking MsgInspector Interface Update.

CATEGORY Compatibility

LOCATION Global

REVIEW Round 1 - Main Review

DESCRIPTION

The old IOAppMsgInspector interface defined an inspect function with the signature inspect(bytes calldata _message, bytes calldata _options), however the new IMsgInspector interface defined for Nexus and Console exposes only a inspect(address _sender, bytes calldata _message, bytes calldata _options) signature. As a result old message inspector implementations will not be compatible with the Nexus and Console.

RECOMMENDATION

Be aware of this breaking change and clearly document it for those building on Nexus and Console.

RESOLUTION

Status: Acknowledged.

INFO I-34

ACKNOWLEDGED

Mints Can Occur While Token Is Paused.

CATEGORY Documentation

LOCATION NexusERC20.sol

REVIEW Round 1 - Main Review

DESCRIPTION

The mint function states that It does not revert if the recipient is not allowlisted, as funds cannot be debited in that state, however as a byproduct of ignoring the checkTransfer validation the mint function also ignores the pause status of the underlying token. This could be intentional to not hold up IzReceive actions and allow them to be executed, however it is important to note that mints can still occur, through receiving crosschain messages or other arbitrary mints by the MINTER_ROLE holders.

RECOMMENDATION

Be aware of this gap in pause validation and consider if it is expected for all cases in which mint can be called. Consider if the isPaused function on the Pause contract should be used to particularly check if the pause status should be consulted before allowing certain mint actions.

RESOLUTION

Status: Acknowledged.

INFO I-35

ACKNOWLEDGED

Payouts Spend Fees From Same Pool.

CATEGORY Warning

LOCATION OFTNative.sol, OFTLockUnlock.sol

REVIEW Round 1 - Main Review

DESCRIPTION

OFTNative and OFTLockUnlock accrue fees in feeBalances, but recipient credits are paid from the same native token pool. Because fees are not reserved separately, bridge payouts can consume balance that accounting still treats as withdrawable fees. In unsupported configurations where remote supply is not fully backed by locked assets, tokens can be bridged back and drain the pool below the tracked fee balance. withdrawFees() may then revert even though fees were previously accrued.

RECOMMENDATION

Use only configurations where remote token supply remains properly backed and cannot dilute the balances relied on to pay out accrued fees.

RESOLUTION

Status: Acknowledged.

INFO I-36

PARTIALLY RESOLVED

Scale Decimals Are Not Exposed.

CATEGORY Compatibility

LOCATION Global

REVIEW Round 1 - Main Review

DESCRIPTION

In both the Nexus and Console implementations that use the RateLimiterUpgradeable, the scaleDecimals are unused and not able to be configured in the constructor. Since SCALE_DECIMALS is an immutable value it's not possible to configure a nonzero scale decimals for any token using Console or Nexus out of the box. Rate limit amounts are applied on a local decimals basis, so any tokens which may reasonably have over `type(uint96).max` (notably tokens with a totalSupply over 79 Billion tokens and 18 decimals) cannot employ the necessary scaling out of the box.

RECOMMENDATION

Consider exposing the scaleDecimals through the constructor to support such tokens out of the box.

RESOLUTION

Status: Partially Resolved.

INFO I-37

ACKNOWLEDGED

quoteOFT Does Not Consult Guard.

CATEGORY Unexpected Behavior LOCATION Nexus.sol REVIEW Round 1 - Main Review

DESCRIPTION

The quoteOFT function in Nexus does not consult the token guard to check if the caller is allowlisted for the token in the first place.

RECOMMENDATION

Consider if quoteOFT should take into account the status of the caller in the guard.

RESOLUTION

Status: Acknowledged.

INFO I-38

ACKNOWLEDGED

Initialize Must Be Invoked During Deployment.

CATEGORY Best Practices

LOCATION Global

REVIEW Round 1 - Main Review

DESCRIPTION

All upgradeable contracts in this repo rely on `initialize(...)` for ownership and critical configuration. If a proxy instance is deployed without calling `initialize` in the same transaction, any external account can front-run and call `initialize` first, seizing ownership/admin roles and permanently compromising the instance.

RECOMMENDATION

Be sure to initialize every deployment in the same transaction to avoid any front-running initialization risks.

RESOLUTION

Status: Acknowledged.

INFO I-39

ACKNOWLEDGED

Nexus Doesn't Support Native Inner Tokens.

CATEGORY Suggestion

LOCATION Nexus.sol

REVIEW Round 1 - Main Review

DESCRIPTION

Console contracts can support native tokens as the underlying token for the OFT, however Nexus contracts do not expose a variant to support any native tokens as they only contain the min/burn variant.

RECOMMENDATION

Consider if this is intended and whether Nexus should be able to support a native token (ERC20 form or canonical native) as an inner token similarly to how the OFTLockUnlock and OFTNative versions of the Console OFT can.

RESOLUTION

Status: Acknowledged.

INFO I-40

ACKNOWLEDGED

New Network Supports Allows Circumvention.

CATEGORY Warning

LOCATION Global

REVIEW Round 1 - Main Review

DESCRIPTION

In Nexus the fees, rate limits, and pause state are based upon the nexusId which depends on the destination EID in addition to the tokenId. As a result, when a new peer is configured for the Nexus for a new Eid there is an opportunity to bypass all of the existing network exit pathway fees, pause state, and rate limits by sending to this new destination EID before it's fee, pause state, and rate limits are configured for each of the tokenIds.

RECOMMENDATION

Be sure to configure the pause state, fees, and rate limits for every token as necessary on a Nexus before configuring a new EID for the Nexus.

RESOLUTION

Status: Acknowledged.

INFO I-41

ACKNOWLEDGED

Lacking Pause Idempotency Check.

CATEGORY Best Practices

LOCATION PauseCore.sol, PauseCoreUpgradeable.sol

REVIEW Round 1 - Main Review

DESCRIPTION

In the PauseCore and PauseCoreUpgradeable contracts, the `_setDefaultPaused` function includes an idempotency check to validate that the configuration being made actually changes state, however no such validation exists for the configurations that are made in the `_setPaused` function for the individual pause configurations.

RECOMMENDATION

Consider introducing an idempotency check in the `_setPaused` function to match the validation performed in the `_setDefaultPaused` function.

RESOLUTION

Status: Acknowledged.

INFO I-42

ACKNOWLEDGED

Event Log Manipulation With Reentrancy.

CATEGORY Events LOCATION OFTNative.sol REVIEW Round 1 - Main Review

DESCRIPTION

OFTNative._IzReceive() calls _credit() before emitting OFTReceived. Because _credit() can transfer native value to a recipient that re-enters, another IzReceive or send can execute before the original receive path emits its completion events. This does not appear to break core accounting, but it can cause event consumers and indexers to observe OFTReceived and PacketDelivered in an order that does not match the order in which operations actually completed.

RECOMMENDATION

Document this ordering caveat so integrators consuming OFTReceived and PacketDelivered do not assume strict completion order from event sequence alone.

RESOLUTION

Status: Acknowledged.

INFO I-43

RESOLVED

NexusAlt Lacks Sanity Check.

CATEGORY Validation

LOCATION NexusAlt.sol

REVIEW Round 1 - Main Review

DESCRIPTION

The other Alt contracts verify that this is indeed a network which uses an ERC20 representation of the native token by validating that the nativeToken result from the endpoint is nonzero. However the NexusAlt contract, which does not need this token directly but is still intended only for these networks, does not implement such a validation.

RECOMMENDATION

Consider implementing sanity validation in the NexusAlt constructor that the `ENDPOINT.nativeToken()` result is nonzero to verify that this is indeed an Alt network.

RESOLUTION

Status: Resolved.

INFO I-44

RESOLVED

Missing MinterBurnerCallFailed Implementation.

CATEGORY Superfluous Code

LOCATION OFTBurnMint.sol

REVIEW Round 1 - Main Review

DESCRIPTION

The OFTBurnMint contract defines a MinterBurnerCallFailed error and stipulates that it is Thrown when a low-level call to MINTER_BURNER fails. However this error is not thrown when a low-level call to the MINTER_BURNER fails and is not used at all.

RECOMMENDATION

Consider removing the MinterBurnerCallFailed error.

RESOLUTION

Status: Resolved.

INFO I-45

ACKNOWLEDGED

Mint/Burn Result Ignored By OFTBurnMint.

CATEGORY Compatibility

LOCATION OFTBurnMint.sol

REVIEW Round 1 - Main Review

DESCRIPTION

The OFTBurnMint implementation intends to support arbitrary mint and burn functions so long as they accept an address followed by a uint256 as parameters. However the OFTBurnMint contract is not compatible with mint/burn functions that fit this signature and return false on failure rather than reverting. This is because the `_callMinterBurner` function simply delegates to the OpenZeppelin `functionCall` function which verifies the success of the call but does not assume or assert anything about the returndata contents.

RECOMMENDATION

Either explicitly document this or support asserting a returned boolean when there is nonzero returndata present.

RESOLUTION

Status: Acknowledged.

INFO I-46

ACKNOWLEDGED

Credit Can Consume An Unexpected Amount Of Gas.

CATEGORY DoS LOCATION OFTNative.sol: 121 REVIEW Round 1 - Main Review

DESCRIPTION

Because the OFTNative automatically loads in the returnData into memory during _credit, this function is susceptible to gas griefing or an OOG when a large amount of bytes are returned: (bool success, bytes memory data) = payable(_to).call{ value: _amountLD }(""); The arbitrary call is also forwarded the maximum available 63/64 amount of gas in the current transaction. Which gives the to address the ability to expend more gas than desired.

RECOMMENDATION

Consider implementing a low level assembly call and specifying 0 as the length of return data to copy into memory to avoid any returndata bombs. Furthermore consider capping the gasLimit that is forwarded to the to address to avoid any unexpected expenditures.

RESOLUTION

Status: Acknowledged.

INFO I-47

ACKNOWLEDGED

Unsynchronized Limits Can Lead To Stuck Funds.

CATEGORY Warning

LOCATION Global

REVIEW Round 1 - Main Review

DESCRIPTION

The configured rate limits can be different between chains where the Oapp contracts are deployed. If the source chain allows sending more than the destination allows for receiving, transactions with an amount between these two limits can be sent but cannot be received on the destination chain because the corresponding transactions to `IzReceive()` will revert.

RECOMMENDATION

Clearly document this risk to users.

RESOLUTION

Status: Acknowledged.

INFO I-48

ACKNOWLEDGED

Rate Limit Bypass Through Exempt Address.

CATEGORY Warning

LOCATION RateLimiterCoreUpgradeable.sol: 302

REVIEW Round 1 - Main Review

DESCRIPTION

The address-exemption feature checks `msg.sender`, not the beneficial token owner. If an exempt router, aggregator, or similar address is allowed to call `send()`, non-exempt users can route through it and bypass rate limits for that destination. The same issue can extend to exempt EOAs using mechanisms like EIP-7702. Once an exempt caller executes the transfer path, all users behind that caller inherit the exemption in practice.

RECOMMENDATION

Treat exempt addresses as high trust, monitor how they are used, and remove exemptions if they allow third parties to bypass intended rate-limit controls.

RESOLUTION

Status: Acknowledged.

INFO I-49

ACKNOWLEDGED

Fee-on-Transfer/Rebasing Tokens Not Supported.

CATEGORY Warning LOCATION Global REVIEW Round 1 - Main Review

DESCRIPTION

On credit, the contract transfers `_amountLD` to the recipient without accounting for tokens that charge a transfer fee/deflate on transfer. The recipient will receive less than `_amountLD` while the protocol still considers the message fully delivered, creating an unaccounted shortfall between the escrowed amount and the actual tokens received.

RECOMMENDATION

Disallow fee-on-transfer and rebasing tokens.

RESOLUTION

Status: Acknowledged.

INFO I-50

ACKNOWLEDGED

Tokens With Inconsistent Decimals Incompatible.

CATEGORY Compatibility

LOCATION Global

REVIEW Round 1 - Main Review

DESCRIPTION

Some tokens have different decimals depending on the network being used such as USDT which has 6 decimals on most chains but 18 decimals on BNB. These tokens are incompatible with the Nexus because there is a requirement that all tokens in the nexus have the same local decimals on all networks the Nexus is deployed on.

RECOMMENDATION

Be aware of this constraint when allowing tokens to be a part of a Nexus. Alternatively, consider supporting solutions for differences in local decimals across tokens on various networks since they may not always be aligned on every network.

RESOLUTION

Status: Acknowledged.

INFO I-51

ACKNOWLEDGED

OFTNative Stuck Message Risk.

CATEGORY Warning LOCATION OFTNative.sol REVIEW Round 1 - Main Review

DESCRIPTION

The OFTNative has a risk of crosschain messages being stuck if they were to target a to address which is a contract that does not accept ether. In this case the message would be stuck in the message channel and there would be no way to rescue that Ether.

RECOMMENDATION

Document this risk and consider performing some initial offchain validations in the UI to protect users from such a risk.

RESOLUTION

Status: Acknowledged.

INFO I-52

RESOLVED

Missing Events.

CATEGORY Events LOCATION Global REVIEW Round 1 - Main Review

DESCRIPTION

Events are missing in the following functions that could benefit from them: •
RateLimiterCoreUpgradeable._setRateLimitGlobalConfig • FeeCoreUpgradeable._accrueFee

RECOMMENDATION

Consider implementing the appropriate events for these cases.

RESOLUTION

Status: Resolved.

INFO I-53

ACKNOWLEDGED

Rate Limit Quotes Can Be Misleading.

CATEGORY Unexpected Behavior

LOCATION Global

REVIEW Round 1 - Main Review

DESCRIPTION

The quote functions only focus on the rate limits for the sending chain and cannot include any information about the rate limit on the receiving chain, which may be misleading for large transfers. There may be a scenario where a transfer is wholly larger than the limit on a receiving chain and can in fact never fit within the limit, in which case the crosschain message would be stuck until the rate limit is raised just for this message.

RECOMMENDATION

Document this gap in the current quoting functions.

RESOLUTION

Status: Acknowledged.

INFO I-54

ACKNOWLEDGED

Outside Tokens Do Not Follow The Guard.

CATEGORY Compatibility

LOCATION Global

REVIEW Round 1 - Main Review

DESCRIPTION

In the Nexus setup when an outside token is used which does not consult the NexusERC20Guard before actions, there can be a divergence between the allowlist/blacklist status of the normal NexusERC20 tokens and this outside token. For example, if a Nexus were to be created where one inner token was USDT while the rest were all NexusERC20 tokens, User A could be blacklisted by the NexusERC20 tokens in the guard but USDT may not have this address blacklisted. Because of this, User A may still interact with the Nexus contract through USDT usage.

RECOMMENDATION

Consider if this setup where an outside token is valid, or if only NexusERC20 tokens can be used with a Nexus hub and document this accordingly.

RESOLUTION

Status: Acknowledged.

INFO I-55

ACKNOWLEDGED

Fee Rounds Down.

CATEGORY Rounding LOCATION FeeCore.sol REVIEW Round 1 - Main Review

DESCRIPTION

In the getFee function, the resulting fee is rounded down in favor of the user and against the favor of the protocol. This is typically against best practices of rounding in protocols, though the material affect is minimal in practice especially due to the fact that the subsequent de-dusting is counted as a fee.

RECOMMENDATION

Confirm if it is acceptable to LayerZero to round the fee amount down in favor of the user in this non-material way.

RESOLUTION

Status: Acknowledged.

INFO I-56

ACKNOWLEDGED

Quote Misses Underlying Token Pause State.

CATEGORY Unexpected Behavior

LOCATION Nexus

REVIEW Round 1 - Main Review

DESCRIPTION

The quoteOFT function on the NexusOFT relies entirely on the nexusQuoteOFT function from the Nexus contract. The nexusQuoteOFT function accounts for the if a particular pathway corresponding to the nexusId or if the Nexus as a whole is paused when computing the oftLimit. However the nexusQuoteOFT function does not account for if the underlying NexusERC20 token is paused. In the NexusERC20Guard contract the checkTransfer function asserts an _assertNotPaused validation for the token, however this is not reflected in the quote result.

RECOMMENDATION

Consider if the pause status of the underlying token should be taken into account when quoting and providing the oftLimit similarly to the individual pathway pauses that may occur in the Nexus contract.

RESOLUTION

Status: Acknowledged.

INFO I-57

ACKNOWLEDGED

Fees Inflated For Small Sends.

CATEGORY Rounding

LOCATION Nexus.sol

REVIEW Round 1 - Main Review

DESCRIPTION

_debitView() applies _removeDust() after subtracting fees and does not refund value rounded away by dust removal. For very small sends, that can make the effective fee far larger than the configured fee rate. For example, after fee subtraction a send may round down to zero received tokens, effectively charging a 100% fee, or lose half its value when the remaining amount is rounded down to the nearest shared-decimal unit. The issue is most noticeable on low-value transfers.

RECOMMENDATION

If this behavior is undesirable, distinguish true fee from dust removed for rounding and refund the de-dusted amount instead of treating it as part of the sent value.

RESOLUTION

Status: Acknowledged.

INFO I-58

ACKNOWLEDGED

Rate Limits Retroactively Affected.

CATEGORY Unexpected Behavior

LOCATION RateLimiterCoreUpgradeable.sol

REVIEW Round 1 - Main Review

DESCRIPTION

In the `_setRateLimitConfigs` function the protocol is careful to checkpoint the rate limit states before updating the details of the decay slope. However this checkpointing is only done for rate limits when their specific config is being updated. Commonly, configs will actually rely on the default config, which uses id 0, for the window and limit of their rate limit. If the default config is updated then all of the rate limit states for those rate limits relying on it will be affected in a stepwise manner retroactively.

RECOMMENDATION

Be aware of this behavior when updating the global rate limit using id 0. Consider manually processing all of the rate limits that rely on it before the global rate limit when passing the list of `SetRateLimitConfigParam`'s.

RESOLUTION

Status: Acknowledged.

INFO I-59

ACKNOWLEDGED

Net Accounting Abused For One Sided Rate.

CATEGORY Rounding

LOCATION RateLimiterCoreUpgradeable.sol

REVIEW Round 1 - Main Review

DESCRIPTION

When `forwardEnabled` is false, `backwardEnabled` is true, and `netAccountingEnabled` is true, the system allows back-and-forth transfers to net usage. Because amounts are rounded up even when reducing usage, an actor can lower current usage more than intended and regain additional capacity. By splitting return transfers into smaller chunks, the actor can repeatedly round reductions in their favor and end up with less recorded usage than before the cycle. That weakens one-sided rate limits beyond the intended netting behavior.

RECOMMENDATION

Round up when increasing `forwardUsage`, but round down when decreasing the opposite-direction usage through net accounting.

RESOLUTION

Status: Acknowledged.

INFO I-60

ACKNOWLEDGED

Rate Limit Configurations Sandwiched.

CATEGORY Frontrunning

LOCATION RateLimiterCoreUpgradeable.sol

REVIEW Round 1 - Main Review

DESCRIPTION

Rate-limit configuration changes take effect immediately. When `useGlobalStateFlag` is enabled, per-path limits are replaced in one transaction by the shared global limiter, and `_setRateLimitStates` can also set exact usage values directly. That creates a sandwich opportunity around the admin transaction: an actor can consume allowance before the update, then back-run after the new configuration lands and consume additional capacity that operators may not have intended to expose within the same block window.

RECOMMENDATION

Make sensitive rate-limit changes through a private mempool or another protected execution path, and account for sandwich exposure when changing limiter configuration.

RESOLUTION

Status: Acknowledged.

INFO I-61

ACKNOWLEDGED

Incorrect To Address Used.

CATEGORY Logical Error

LOCATION NexusOFT.sol

REVIEW Round 1 - Main Review

DESCRIPTION

In the Nexus contract, when a credit is made to the zero address the address is re-assigned to the dead address in the `_credit` function to avoid reverts with some underlying ERC20 implementations. However the `NexusOFT.nexusReceive` function receives the original to address as the `_to` parameter, which has not been remapped to the dead address. In this case the `OFTReceived` event and the `sendCompose` call will use the incorrect receiver address, as it will present as `address(0)` but actually be the dead address.

RECOMMENDATION

Re-map `address(0)` to the dead address in the `nexusReceive` function of the `NexusOFT` contract.

RESOLUTION

Status: Acknowledged.

INFO I-62

ACKNOWLEDGED

Default Rate Limit Cannot Apply Across Tokens.

CATEGORY Compatibility

LOCATION RateLimiterCoreUpgradeable.sol

REVIEW Round 1 - Main Review

DESCRIPTION

The RateLimiterCoreUpgradeable contract includes a default rate limit that is intended to be shared across tokens supported by the Nexus. The Nexus does require that these tokens all use the same decimals and shared decimals, however it does not make sense for this default rate limit to apply across assets that have different prices. For example an asset with a price of \$100,000 per 1e18 tokens cannot abide by the same rate limit as an asset with a price of \$1 per 1e18 tokens.

RECOMMENDATION

Consider optionally allowing a default rate limit to be defined by a USD value of tokens being moved via an arbitrary supported oracle pricing solution. Otherwise, be aware of this deficiency in the ability to enforce a default rate limit in a valuable way, and clearly document this for those who use Nexus to rate limit multiple tokens.

RESOLUTION

Status: Acknowledged.

INFO I-63

ACKNOWLEDGED

nexusQuoteOFT Misses Exemptions.

CATEGORY Warning LOCATION Nexus.sol REVIEW Round 1 - Main Review

DESCRIPTION

The nexusQuoteOFT function uses the maxAmountLD as presented by the getRateLimitUsages function. However this function does not account for when the sender who would initiate an OFT send is exempt when presenting the available amount. The nexusQuoteOFT function provides no extra logic to raise the available amount to the type(uint256).max if the ultimate caller of the quoteOft function on the NexusOFT contract is exempt, and therefore portrays an inaccurate quote result for that caller.

RECOMMENDATION

Consider taking into consideration whether the caller of the quoteOFT function on the NexusOFT contract is exempt or not to portray an accurate quote response in the nexusQuoteOFT function. Note that this vector also applies to Console, hence quoteOFT can check for exemptions there as well.

RESOLUTION

Status: Acknowledged.

INFO I-64

ACKNOWLEDGED

nexusQuoteOFT Does Not Consult The MsgInspector.

CATEGORY Unexpected Behavior

LOCATION Nexus.sol

REVIEW Round 1 - Main Review

DESCRIPTION

In the `nexusQuoteOFT` function, the `msg` inspector is not consulted to verify if the `send` may be invoked or not based upon the sender, `sendParams`, and options. However, the `nexusQuoteSend` function does consult the `msg` inspector by way of using the `_buildMsgAndOptions` function.

RECOMMENDATION

Consider if the `nexusQuoteOFT` function ought to consult the `msg` inspector to more accurately mimic the result of a `send` operation. Or consider standardizing both the `nexusQuoteOFT` and `nexusQuoteSend` functions to both not consult the `msg` inspector if it should not be considered. Otherwise clearly document this difference between the two quote functions.

RESOLUTION

Status: Acknowledged.

INFO I-01

ACKNOWLEDGED

Stale Checkpointing Documentation.

CATEGORY Documentation LOCATION IRateLimiter.sol REVIEW Round 2 - Remediation Review

DESCRIPTION

IRateLimiter.checkpointRateLimits docs still state it should be used "when not using setRateLimitConfigs", but current behavior is the opposite: setRateLimitConfigs does not checkpoint state. This creates contradictory guidance for operators updating limits/windows.

RECOMMENDATION

Update the checkpointRateLimits docstring to reflect current behavior, e.g. "call before changing limits/windows..." so it is clear that rate limiter admin must trigger checkpoints, otherwise changes can apply retroactively.

RESOLUTION

Status: Acknowledged.

INFO I-02

ACKNOWLEDGED

Potential Overflow In maxAmountLD.

CATEGORY Warning LOCATION Nexus.sol REVIEW Round 2 - Remediation Review

DESCRIPTION

In `nexusQuoteOFT`, `maxAmountLD` is rescaled by fee basis points: $\text{maxAmountLD} = (\text{maxAmountLD} * \text{BPS_DENOMINATOR}) / (\text{BPS_DENOMINATOR} - \text{feeBps})$; This is currently safe because available amounts are tightly bounded, the `uint256`.max sentinel is excluded before the multiplication, and the full-fee case is handled separately. The concern is that this safety depends on those invariants remaining true. If future changes introduce scaled availability or broader bounds, this multiplication path could become an overflow or revert surface.

RECOMMENDATION

Document the assumptions that keep this safe today, and revisit the calculation if future changes expand the range of `maxAmountLD`.

RESOLUTION

Status: Acknowledged.

INFO I-03

ACKNOWLEDGED

AccessControl2StepUpgradeable Lacks Timelock.

CATEGORY Warning

LOCATION AccessControl2StepUpgradeable.sol

REVIEW Round 2 - Remediation Review

DESCRIPTION

AccessControl2StepUpgradeable implements a two-step DEFAULT_ADMIN_ROLE transfer modeled after OpenZeppelin's AccessControlDefaultAdminRules. However, it omits a time delay between beginDefaultAdminTransfer() and acceptDefaultAdminTransfer(). As implemented, the Acknowledged admin can accept the transfer in the very same block as the transfer is initiated without delay.

RECOMMENDATION

Clearly document this behavior or consider adding a delay.

RESOLUTION

Status: Acknowledged.

INFO I-04

ACKNOWLEDGED

PauseByIDBase._setPaused Lacks Idempotent Check.

CATEGORY Warning

LOCATION PauseByIDBase.sol

REVIEW Round 2 - Remediation Review

DESCRIPTION

PauseByIDBaseUpgradeable._setPaused does not check whether the new config matches the existing config before writing. Every other pause setter in the codebase reverts on same-state writes: • PauseBaseUpgradeable._setPaused(bool) - reverts with PauseStateIdempotent • PauseByIDBaseUpgradeable._setDefaultPaused(bool) - reverts with PauseStateIdempotent But PauseByIDBaseUpgradeable._setPaused(SetPausedParam[]) unconditionally overwrites and emits, which allows the same config to be written repeatedly, emitting duplicate PauseSet events each time.

RECOMMENDATION

Consider adding an idempotent check per entry,

RESOLUTION

Status: Acknowledged.

INFO I-05

ACKNOWLEDGED

Missing AccessControl Init Locks Admin Functions.

CATEGORY Logical Error

LOCATION Global

REVIEW Round 2 - Remediation Review

DESCRIPTION

OAppCoreRBACUpgradeable protects management functions such as `setPeer` and `setDelegate` with `onlyRole(OAPP_ADMIN_ROLE)`, but its initializer does not initialize `AccessControl2Step`. If an integrating contract calls only `__OAppCoreRBAC_init(...)` and skips `__AccessControl2Step_init(...)`, no account receives `DEFAULT_ADMIN_ROLE`, `grantRole` cannot be used, and the role-gated admin paths can be locked permanently.

RECOMMENDATION

Update the docs and initialization guidance to state explicitly that inheritors must call `__AccessControl2Step_init(initialAdmin)`, and that omitting this step can lock admin functionality.

RESOLUTION

Status: Acknowledged.

INFO I-06

ACKNOWLEDGED

PreCrime Simulation Layer Entirely Removed.

CATEGORY Warning LOCATION OFTCoreBaseUpgradeable.sol REVIEW Round 2 - Remediation Review

DESCRIPTION

The Console implementation completely removes LayerZero's PreCrime simulation subsystem from the OFT stack. In OFTCoreBaseUpgradeable, the inheritance chain is OAppUpgradeable → OAppOptionsType3BaseUpgradeable → OAppMsgInspectionBaseUpgradeable - OAppPreCrimeSimulatorUpgradeable is absent.

RECOMMENDATION

Consider if this intended. If so, clearly document this behavior.

RESOLUTION

Status: Acknowledged.

INFO I-01

ACKNOWLEDGED

Shared Nonce Channel Enables Token-level DoS.

CATEGORY DoS LOCATION Nexus REVIEW Round 3 - Remediation Review

DESCRIPTION

`setPeer(dstEid, peer)` creates a single shared LayerZero lane between Nexus contracts. At the protocol level, LayerZero identifies channels by (sender, srcEid, dstEid, receiver) and maintains a single sequential nonce per lane, with no awareness of token IDs. Although each token has its own NexusOFT, all tokens route through the same Nexus hub, so transfers for USDC, WBTC, and all other tokens share one nonce stream between chains. On the source side, a send is accepted as long as the OFT is registered locally and a peer exists for the destination chain, but it does not verify that the token is registered on the destination Nexus. This allows a token that exists on chain A but not on chain B to still be sent. The failure only occurs on B inside `_lzReceive()`, where `InvalidTokenId(tokenId)` is triggered. By that point, the

message has already been verified and stored in the LayerZero endpoint, but not cleared. Because LayerZero uses a lazy inbound nonce, clearing later messages requires iterating through all prior nonces in the lane. If invalid packets accumulate ahead of valid ones, this linear scan becomes increasingly expensive and can run out of gas, preventing legitimate transfers from being processed. An attacker can exploit this by repeatedly sending a token supported on A but not B (even with `amountLD = 0`). These messages are accepted on A, revert on B, and remain uncleared, creating a backlog. Since all tokens share the same Nexus lane, this backlog impacts unrelated tokens, eventually causing legitimate transfers to fail due to gas exhaustion during `_clearPayload`. This results in a global DoS across all tokens on the affected chain pair. The attack is cheap to execute (only messaging fees), and

recovery is operational, requiring manual clearing of stuck packets while user funds remain blocked until intervention. <https://gist.github.com/GuardianAudits/6cf23d0d7c36bf1bd299aa4b22da15b8>

RECOMMENDATION

`setPeer(B)` opens the $A \rightarrow B$ road for the whole Nexus `registerToken(X)` decides which tokens exist on each chain The bug needs token X exists on A, does not exist on B, but the $A \rightarrow B$ road is still open for X A mitigation for this can be to pause any affected route, by blocking any exact pair, `tokenId X → destination Eid B`, before `nexusSend()` burns and sends By computing the route ID with `nexus.getNexusId(tokenId, dstEid)`, then calling `setPaused(routeId)` on the pause module

RESOLUTION

Status: Acknowledged.

INFO I-02

ACKNOWLEDGED

Immutable Decimals Can Change On Upgrade.

CATEGORY Upgradeability

LOCATION ERC20Plus.sol:21-92

REVIEW Round 3 - Remediation Review

DESCRIPTION

ERC20Plus stores DECIMALS as an immutable set in the implementation constructor. In a proxy setup, immutables live in the implementation bytecode (not proxy storage). If the proxy is upgraded to an implementation deployed with a different constructor `_decimals`, `decimals()` will silently change while balances/allowances/totalSupply remain in the old units.

RECOMMENDATION

Do not use an immutable for decimals in an upgradeable token. Store decimals in proxy storage (set once in `initialize()`), and enforce that it cannot change across upgrades.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

OFT interface ID omits receiver functions.

CATEGORY Unexpected Behavior

LOCATION apps/oapp-app/contracts/evm/non-upgradeable/contracts/interfaces/IAppExtended.sol#L23-L28

REVIEW Round 4 - Updates Review

DESCRIPTION

`OAPP_EXTENDED_INTERFACE_ID` is intended to flatten the complete OApp interface tree. However, it includes `type(IAppReceiver).interfaceId` without separately including its parent, `ILayerZeroReceiver`. Solidity calculates an interface ID using only functions declared directly by that interface. Consequently, `type(IAppReceiver).interfaceId` includes `isComposeMsgSender` but excludes the inherited `allowInitializePath`, `nextNonce`, and `lzReceive` functions. `/// @dev Flatten all leaf interface IDs.`

```
bytes4 constant OAPP_EXTENDED_INTERFACE_ID = type(IAppReceiver).interfaceId ^
type(IAppMessagingChannel).interfaceId ^ type(IAppCore).interfaceId ^ type(IAppOptionsType3).interfaceId ^
type(IAppMsgInspection).interfaceId;
```

The incomplete OApp ID propagates into `OFT_EXTENDED_INTERFACE_ID`. The contract currently returns `0x97b6b900`, whereas the complete flattened interface ID is `0x06fbb406`. The difference is exactly the omitted `ILayerZeroReceiver` interface ID. Integrations that independently calculate the identifier from the complete ABI may reject or misidentify compatible OFT deployments.

RECOMMENDATION

Include `type(ILayerZeroReceiver).interfaceId` when calculating `OAPP_EXTENDED_INTERFACE_ID`.

RESOLUTION

Status: Resolved.

Remediation validated in `e61f5a7220495a1143f70cc2f1c6bea8a4dad722`.

INFO I-01

ACKNOWLEDGED

Divergent guards can block Nexus credits.

CATEGORY Unexpected Behavior

LOCATION contracts/common/utills/evm/upgradeable/contracts/credit-redirect/CreditRedirectBaseUpgradeable.sol#L63-L84 and related helpers

REVIEW Round 4 - Updates Review

DESCRIPTION

Nexus processes inbound transfers for multiple registered tokens using a single global `CreditRedirectStorage` configuration. However, each registered NexusERC20 stores an independent guard pointer that its administrator can replace through `setGuard`. When an inbound message is executed, Nexus first determines the effective recipient using the global allowlist. `if (!_isAllowlisted(_to)) { /// @dev escrow is guaranteed to be non-zero here. address escrow = _getCreditRedirectStorage().escrow; emit CreditRedirected(_to, escrow, _amountLD); return escrow; } return _to;` If the global allowlist considers the intended recipient eligible, `_redirectCredit` returns the original recipient instead of the escrow address. Next, Nexus invokes the registered NexusERC20's `mint` function. Before minting, NexusERC20 performs another validation using that token's current guard.

If the global allowlist and the token's current guard contain different policies, the global allowlist may accept a recipient that the token guard rejects. In that case, Nexus does not redirect the credit to escrow because `_redirectCredit` returns the original recipient, but the subsequent guard check reverts the mint. Consequently, the sender's tokens have already been burned on the source chain and remain temporarily unavailable while the destination message is pending. Although the intended deployment uses one shared guard for all Nexus tokens, this relationship is not enforced during token registration, credit redirect configuration, or subsequent guard replacement.

RECOMMENDATION

Consider enforcing that Nexus's credit-redirect allowlist remains compatible with every registered NexusERC20 guard.

RESOLUTION

Status: Acknowledged.

Implementers should use the same allowlist for both credit redirection and token allowlisting; otherwise, the behavior will drift.

INFO I-02

RESOLVED

Empty init data leaves proxy unprotected.

CATEGORY Informational

LOCATION apps/proxy-app/contracts/evm/proxy-v5-6-evm/contracts/TransparentUpgradeableProxy2Step.sol#L70-L74

REVIEW Round 4 - Updates Review

DESCRIPTION

TransparentUpgradeableProxy2Step forwards the caller-provided initialization data to the inherited ERC1967Proxy constructor. The pinned OpenZeppelin version permits this data to be empty, in which case the implementation is installed without initializing the proxy's storage. If the implementation exposes an external initializer, it remains callable through the proxy. Because non-admin calls are delegated to the implementation, anyone can invoke the initializer before the legitimate administrator and assign themselves application ownership, roles, or other privileged configuration.

RECOMMENDATION

Require the proxy to be initialized atomically during deployment using the expected encoded initializer call. Consider upgrading to OpenZeppelin Contracts v5.6 or later, which rejects empty initialization data by default.

RESOLUTION

Status: Resolved.

Remediation validated in `0ac5096c55c93a010b681955457868fa860b42c4`.

INFO I-03

RESOLVED

Burn docs omit nonce eligibility.

CATEGORY Documentation

LOCATION apps/oapp-app/contracts/evm/non-upgradeable/contracts/interfaces/I0AppMessagingChannel.sol#L31-L38

REVIEW Round 4 - Updates Review

DESCRIPTION

The interface describes `burn` as removing a verified inbound nonce, implying that any verified packet can be burned with the correct payload hash. However, the Endpoint additionally requires the nonce to be at or below `lazyInboundNonce`. `if (curPayloadHash == EMPTY_PAYLOAD_HASH || _nonce > lazyInboundNonce[_oapp][_srcEid][_sender]) revert Errors.LZ_InvalidNonce(_nonce);` Because verification does not advance `lazyInboundNonce`, burning a freshly verified packet will revert while its nonce remains above the lazy checkpoint. The incomplete documentation may cause operators to submit reverting recovery transactions or misunderstand when `burn` is applicable.

RECOMMENDATION

Document that `burn` requires an exact, nonempty stored payload hash and `_nonce ≤ lazyInboundNonce`.

RESOLUTION

Status: Resolved.

Remediation validated in `e61f5a7220495a1143f70cc2f1c6bea8a4dad722`.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian.

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.